SALSWIFT（以下简称 SF1）MCU 开发环境使用指南

1 术语/缩略词

FPGA	：Field Programmable Gate Array	现场可编程门阵列
TD	：Tang Dynasty	上海安路软件集成开发环境
MCU	：Micro Controller Unit	微型控制单元
JTAG	：Joint Test Action Group	联合测试工作组

2 关于本手册

本手册将对 SF1 系统工程的建立、调试作详细介绍。

3 SF1 MCU 调用与端口描述

3.1 IPGen GUI

在 TD 5.6.1 环境下，打开 IP Generator，选择 SF1 RISC-V，界面如图 3-1 所示。

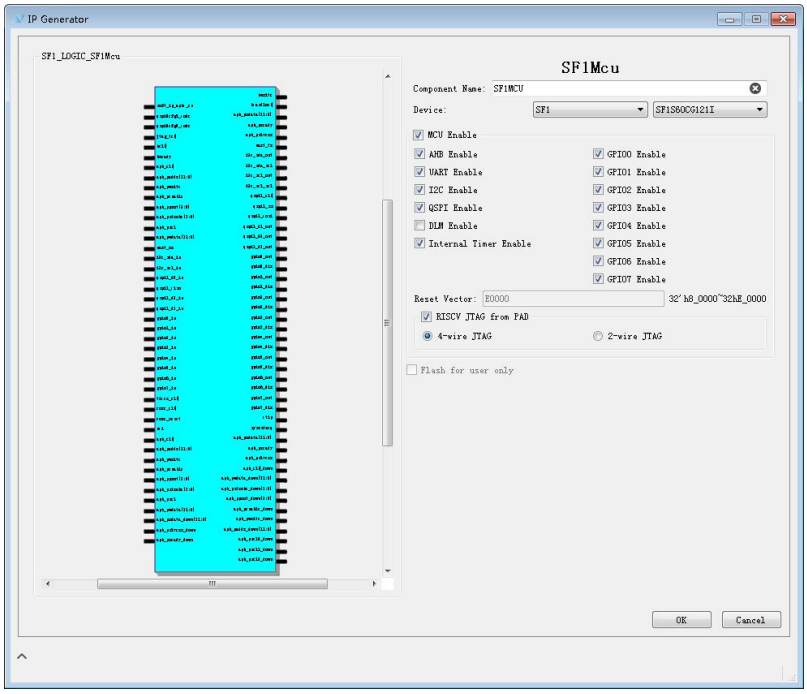

图 3-1 SF1Mcu IP GUI

MCU GUI 相关使能参数，如表 3-1-1 所示。

表 3-1-1 MCU GUI 使能参数

GUI 选项	说明
MCU Enable	MCU 使能，若不使能，预留 APB 部分，搭建 APB Master 配置 DSC/DSI。
AHB Enable	AHB 使能。

UART Enable	UART 使能。
I2C Enable	I2C 使能。
QSPI Enable	QSPI 使能。
TIMER Enable	TIMER 使能。
GPIO0~7 Enable	GPIO0~7 使能。
Reset Vector	Reset Vector，默认 E0000。
4-wire JTAG	4-wire JTAG 使能，TCK/TMS/TDI/TDO。
2-wire JTAG	2-wire JTAG 使能，TCK/TMS。
Flash for user only	用户访问 Flash 使能。

3.2 MCU 端口信号描述

通过 IP Generator 工具例化生成文件，各个端口描述如表 3-1-2 所示。

表 3-1-2 MCU GUI 使能参数

端口名	方向	位宽	描述
soft_ip_apbm_en	输入	1	DSI、DSC 配置接口权限控制 1’b1:使能逻辑配置 DSI、DSC 的 APB 接口通路 1’b0:由 MCU 控制 DSI、DSC 的 APB 配置通路
qspi0cfg1_mode	输入	1	FLASH 权限控制 1’b1:FALSH 访问接口由 MCU 使用 1’b0:FALSH 访问接口由逻辑控制使用
qspi0cfg2_mode	输入	1	FLASH 权限控制（封装不同） 1’b1:FALSH 访问接口由 MCU 使用 1’b0:FALSH 访问接口由逻辑控制使用
core_clk	输入	1	MCU 输入时钟，由 MCU 寄存器控制是否使用
core_reset	输入	1	系统复位，复位 MCU 除 JTAG 以外的所有模块，可以由 FPGA 逻辑产生复位信号。
sysrstreq	输出	1	Debug/SW reset 模式下用于 reset SOC 系统。
timer_clk	输入	1	MCU 计时器模块的低频时钟，可以由 FPGA 内部时钟分频产生。
jtag_TCK	输入	1	JTAG TCK signal.
jtag_TMS_in	输入	1	4-wire JTAG TMS signal 2-wire JTAG TMS input signal from IO PAD.
jtag_TMS_out	输出	1	2-wire JTAG TMS output signal to IO PAD
jtag_DRV_TMS	输出	1	2-wire JTAG TMS output enable signal to IO

			PAD. When the TMS is outputting, this DRV_TMS signal will be high to enable IO PAD as output
jtag_TDI	输入	1	4-wire JTAG TDI signal.
jtag_TDO	输出	1	4-wire JTAG TDO signal.
jtag_DRV_TDO	输出	1	4-wire JTAG TDO output enable signal to IO PAD. When the TMO is outputting, this DRV_TMO signal will be high to enable IO PAD as output.
jtag_dwen	0	1	指示 soc 已经进入二线调试模式
nmi	输入	1	非屏蔽中断
clic_irq	输入	16	FPGA 产生的 MCU 中断输出
dlm_cs	输出	1	SRAM’s Csa signal.
dlm_ byte_we	输出	4	SRAM’s wea signal.
dlm_addr	输出	11	SRAM’s ADDR signal.
dlm_wdata	输出	32	SRAM’s RAM_IN signal.
dlm_rdata	输入	32	SRAM’s RAM_OUT signal.
clk_dlm_ram	输出	1	SRAM’s CLK signal
hclk	输出	1	AHB-Lite protocol’s HCLK signal.
htrans	输出	2	当 soft_ip_apbm_en=0 时 AHB-Lite protocol’s HTRANS signal. 当 soft_ip_apbm_en=1 时 bit[1]:APB protocol’s READY signal bit[0]:APB protocol’s SLVERR signal
hwrite	输出	1	AHB-Lite protocol’s HWRITE signal.
haddr	输出	32	AHB-Lite protocol’s HADDR signal.
hsize	输出	3	AHB-Lite protocol’s HSIZE signal
hburst	输出	3	AHB-Lite protocol’s HBURST signal.
hprot	输出	4	AHB-Lite protocol’s HPROT signal.
hmastlock	输出	1	AHB-Lite protocol’s HLOCK signal.
hwdata	输出	32	AHB-Lite protocol’s HWDATA signal.
hrdata	输入	32	当 soft_ip_apbm_en=0 时 AHB-Lite protocol’s HRDATA signal. 当 soft_ip_apbm_en=1 时 APB protocol’s WDATA signal

hresp	输入	2	当 soft_ip_apbm_en=0 时 AHB-Lite protocol’ s HRESP signal 当 soft_ip_apbm_en=1 时 bit[1]:APB protocol’ s WRITE signal bit[0]:APB protocol’ s ENABLE signal
hready	输入	1	AHB-Lite protocol’ s HREADY signal
ext_apb_clk	输入	1	APB protocol’ s CLK signal
ext_apb_pprot	输入	3	APB protocol’ s PPROT signal
ext_apb_pstrobe	输入	4	APB protocol’ s PSTROBE signal
ext_apb_addr	输入	32	APB protocol’ s ADDR signal
ext_apb_psel	输入	1	APB protocol’ s PSEL signal
uart_tx	输出	1	UART’ s TX signal
uart_rx	输入	1	UART’ s RX signal
i2c_sda_in	输入	1	I2C’ s SDA IN signal
i2c_sda _out	输出	1	I2C’ s SDA OUT signal
i2c_sda _sel	输出	1	I2C’ s SDA SEL signal
i2c_scl_in	输入	1	I2C’ s SCL IN signal
i2c_scl_out	输出	1	I2C’ s SCL OUT signal
i2c_scl_sel	输出	1	I2C’ s SCL SEL signal
qspi1_clk	输出	1	QSPI1’ s CLK signal.
qspi1_ss	输出	1	QSPI1’ s SS signal.
qspi1_mosi	输出	1	QSPI1’ s MOSI/ QSPI1 D0 OUTPUT
qspi1_d0_in	输入	1	QSPI1 D0 INPUT
qspi1_miso	输入	1	QSPI1’ s MISO/ QSPI1 D1 INPUT
qspi1_d1_out	输出	1	QSPI1 D1 OUTPUT
qspi1_d2_in	输入	1	QSPI1 D2 INPUT
qspi1_d2_out	输出	1	QSPI1 D2 OUTPUT
qspi1_d3_in	输入	1	QSPI1 D3 INPUT
qspi1_d3_out	输出	1	QSPI1 D3 OUTPUT
qspi1_dir	输出	4	QSPI1 Direction selection signal.
gpio0_in	输入	1	GPIO0 INPUT
gpio0_out	输出	1	GPIO0 OUTPUT
gpio0_dir	输出	1	GPIO0 direction
gpio1_in	输入	1	GPIO1 INPUT
gpio1_out	输出	1	GPIO1 OUTPUT
gpio1_dir	输出	1	GPIO1 direction

gpio2_in	输入	1	GPIO2 INPUT
gpio2_out	输出	1	GPIO2 OUTPUT
gpio2_dir	输出	1	GPIO2 direction
gpio3_in	输入	1	GPIO3 INPUT
gpio3_out	输出	1	GPIO3 OUTPUT
gpio3_dir	输出	1	GPIO3 direction
gpio4_in	输入	1	GPIO4 INPUT
gpio4_out	输出	1	GPIO4 OUTPUT
gpio4_dir	输出	1	GPIO4 direction
gpio5_in	输入	1	GPIO5 INPUT
gpio5_out	输出	1	GPIO5 OUTPUT
gpio5_dir	输出	1	GPIO5 direction
gpio6_in	输入	1	GPIO6 INPUT
gpio6_out	输出	1	GPIO6 OUTPUT
gpio6_dir	输出	1	GPIO6 direction
gpio7_in	输入	1	GPIO7 INPUT
gpio7_out	输出	1	GPIO7 OUTPUT
gpio7_dir	输出	1	GPIO7 direction
mtip	输出	1	Timer notice
apb_clk	输出	1	Used to configure DSI/DSC,HARD WIRE
apb_prdata	输入	32	Used to configure DSI/DSC,HARD WIRE
apb_pwdata	输出	32	Used to configure DSI/DSC,HARD WIRE
apb_pstrobe	输出	4	Used to configure DSI/DSC,HARD WIRE
apb_pprot	输出	3	Used to configure DSI/DSC,HARD WIRE
apb_penable	输出	1	Used to configure DSI/DSC,HARD WIRE
apb_pwrite	输出	1	Used to configure DSI/DSC,HARD WIRE
apb_pslverr_i	输入	1	Used to configure DSI/DSC,HARD WIRE
apb_pready_i	输入	1	Used to configure DSI/DSC,HARD WIRE
apb_paddr_o	输出	12	Used to configure DSI/DSC,HARD WIRE
apb_psel0_o	输出	1	Used to configure DSI/DSC,HARD WIRE
apb_psel1_o	输出	1	Used to configure DSI/DSC,HARD WIRE
apb_psel2_o	输出	1	Used to configure DSI/DSC,HARD WIRE

4 SF1 工程建立

4.1 环境准备与安装

下载 TD 软件及其 AnlogicStudio 安装包，TD 安装可参考《TD User Guide》，解压 AnlogicStudio 后，可直接双击 AnlogicStudio.exe 启动开发环境。

其中硬件开发环境如下：

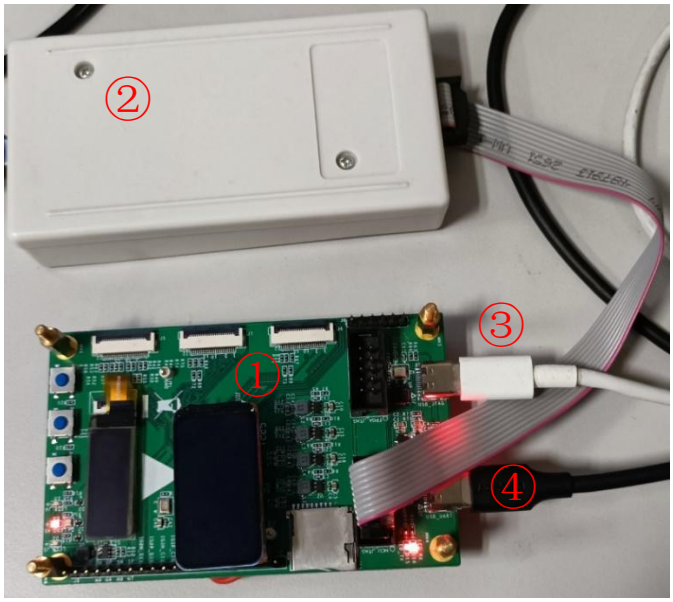

图 4-1 SF1 硬件开发环境

图处：SF1 开发板；

图处：MCU 调试器；

图处：Type C 供电端口、板子 FPGA JTAG 下载器；

图处：Type C USB 串口连接端口；

开发环境：TD 版本为 TD5.6.1，Eclipse 版本为 2022.4。

4.2 TD 工程创建

打开 TD 5.6.1 软件，通过新建工程向导，建立 SF1 FPGA 工程，在 device family 选择 SF1 器件，如图 4-2 所示。

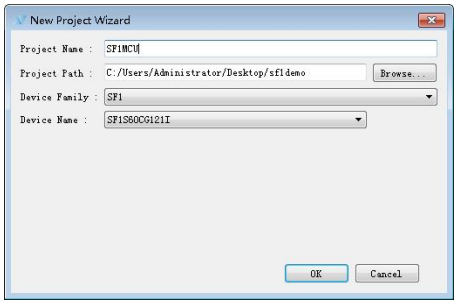

图 4-2 工程向导

在菜单栏中选择 IP Generator，选择 SF1-RISCV，可根据需求对 MCU 外设、启动地址进行配置，如图 4-3、4-4 所示。

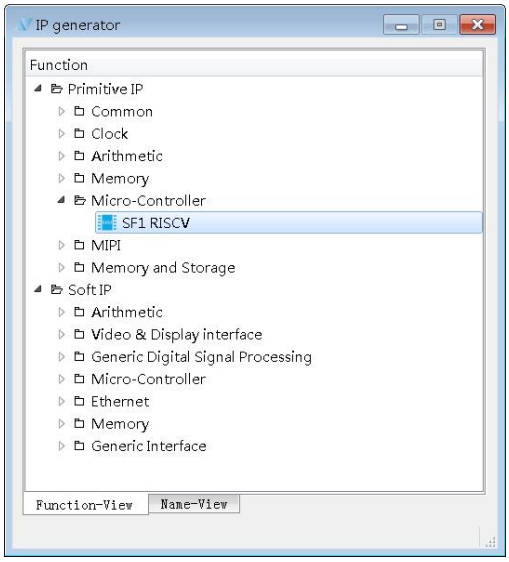

图 4-3 IP Generator GUI

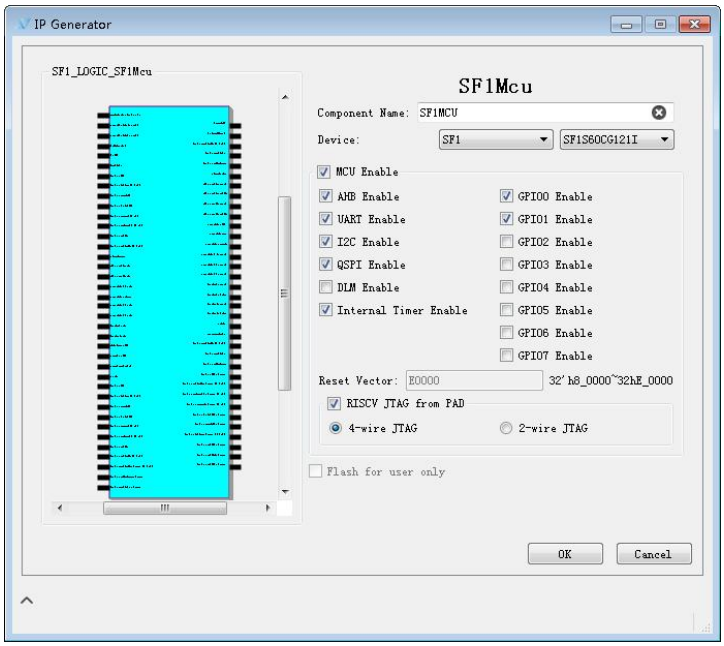

图 4-4 SF1MCU IP 配置界面

通过 IPGenerator 生成 SF1 MCU，增加相关逻辑，工程如图 4-5 所示。

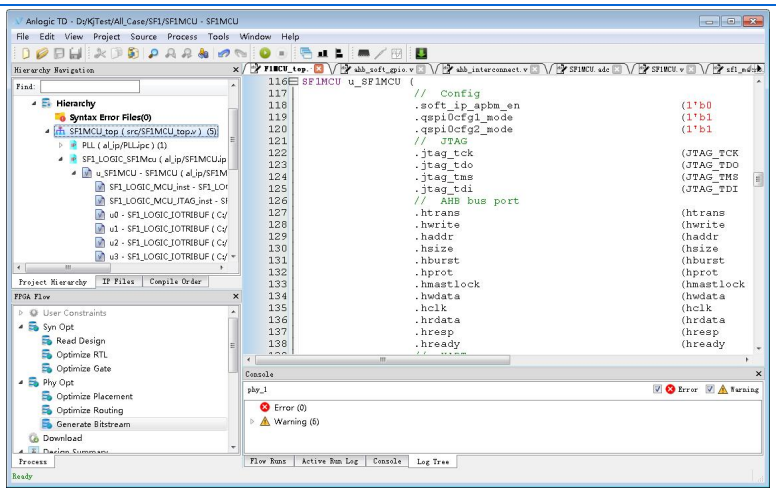

图 4-5 TD 工程

4.3 AnlogicStudio 工程创建与调试

打开 AnlogicStudio，如图 4-6 所示，选择 Workspace 后，进入 Eclipse IDE 环境。在调试前，需确认 SF1 开发板及其下载器工作正常，且已烧写 TD 工程对应的位流文件。

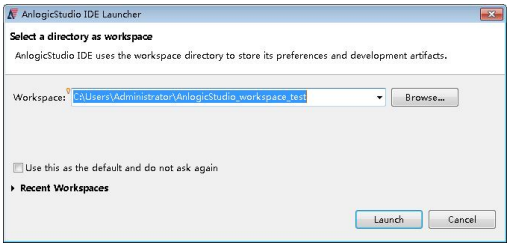

图 4-6 Eclipse Workspace

通过 File->New Project->C/C++->C Project，创建 C 工程

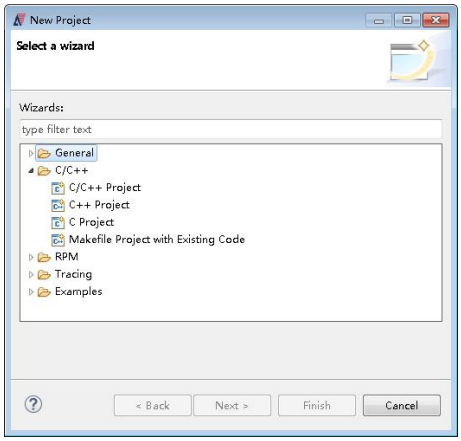

图 4-7 C Project

选择 Project Type，输入 Project Name：

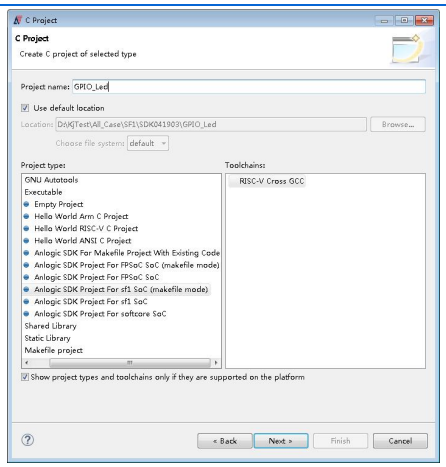

图 4-8 SF1 Project

选择 Project Type，输入 Project Name，如图 4-9 所示，运行模式根据需求可选择 work1/2/3。

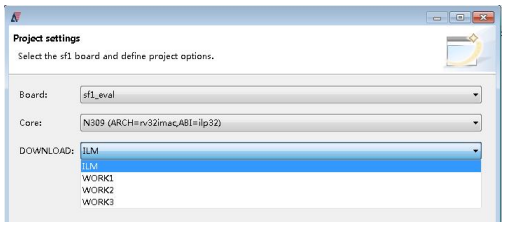

图 4-9 Project Setting

选择 Project Example，选择 GPIO_Led，其他可用默认，如图 4-10、4-11 所示。

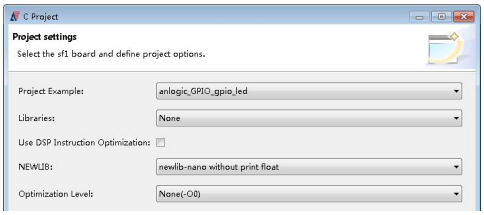

图 4-10 Project Setting

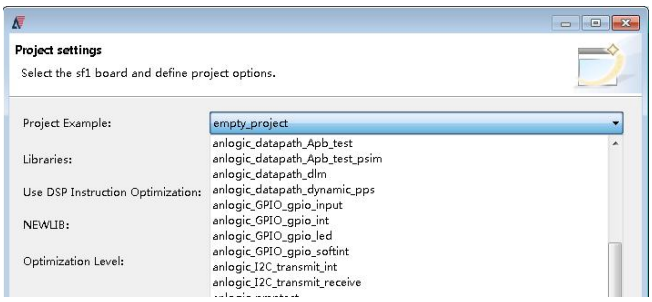

图 4-11 SDK Example

按照上述步骤建立工程，后续页面默认配置，点击 Next、Finish 后，直至完成工程建立，如图 4-12 所示。

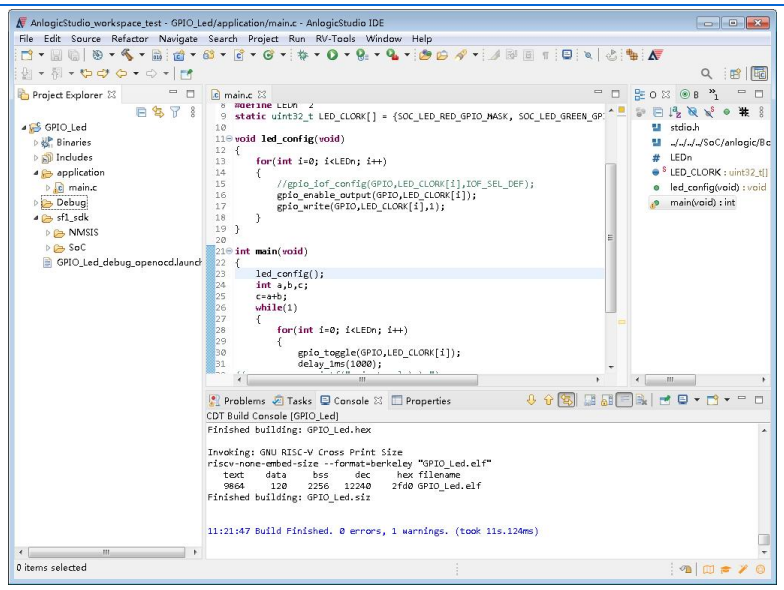

图 4-12 工程建立

SF1 Makefile 项目配置：Project->Properties->C/C++ Build->Builder Settings，取消 Generate Makefiles automatically 勾选，将 Build directory 路径 ${workspace_loc:/xxx}/Debug 修改为 ${workspace_loc:/xxx}，如图 4-13 所示，点击 Apply and Close。选择 Project->Build All，编译通过后，在 Build 目录下，生成相关文件。

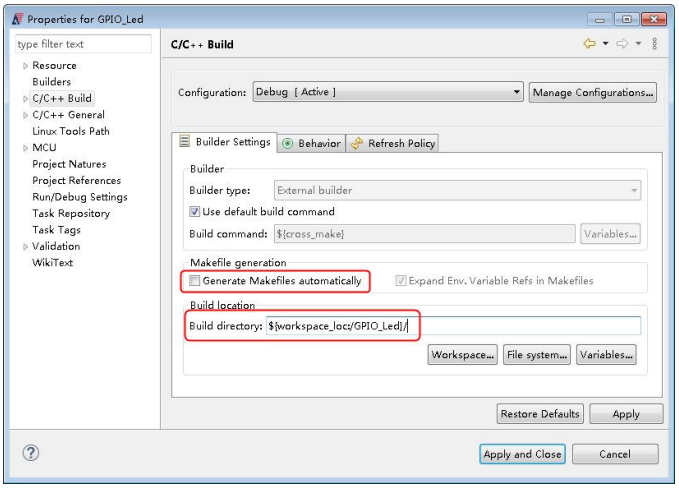

图 4-13 Builder Setting

调试前，如若未安装下载器驱动，可在 AnlogicStudio 文件夹下，打开 usb driver tool，路径如下:AnlogicStudio\toolchain\softcore\openocd\drivers\UsbDriverTool，将 USB Serial Converter A 的驱动更改为 Install Libusb -WinUSB，如图 4-14 所示。

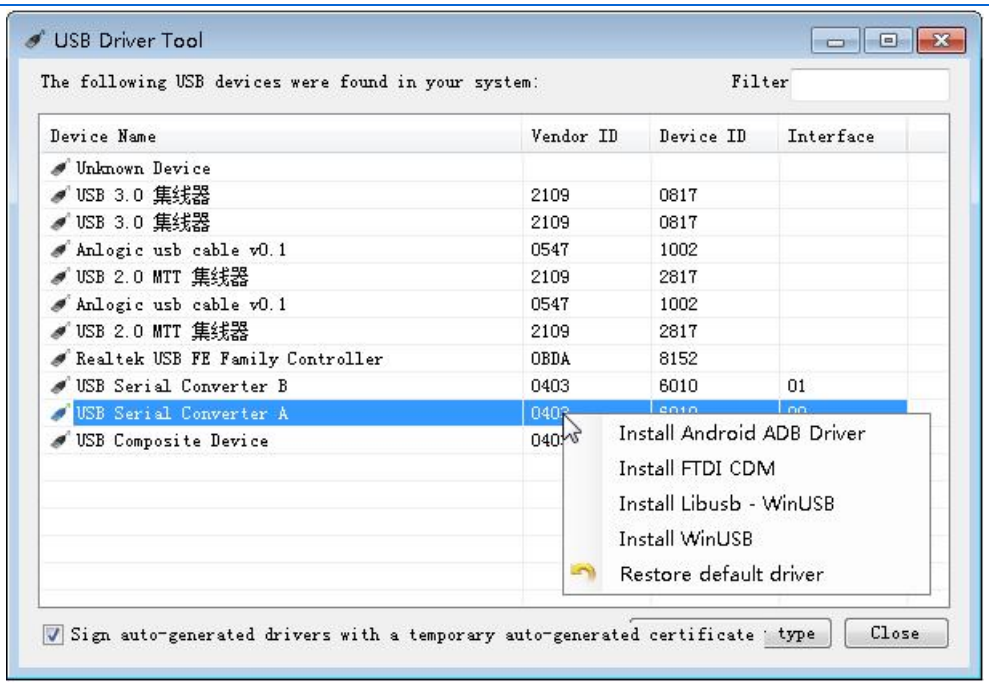

图 4-14 驱动安装

单击工程右键 Run As/Debug As->Run Configuration/Debug Configuration，选择 GPIO_Led_debug，实现在线 RUN/Debug，如图 4-15 所示。

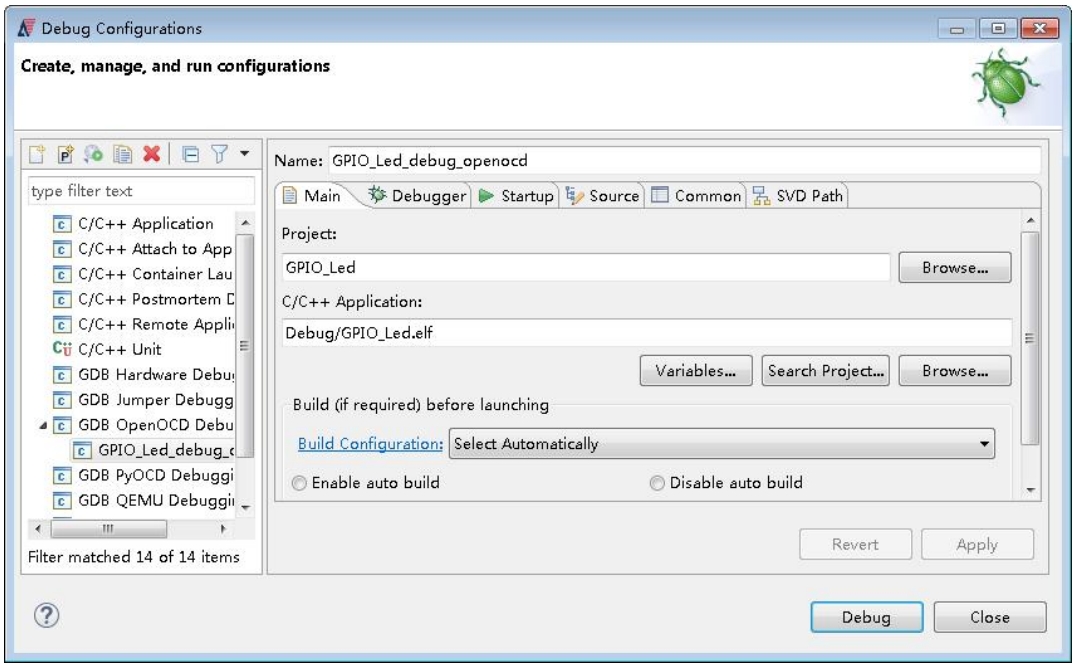

图 4-15 Debug Configurations

进入 Debug 调试界面，对程序可实现单步调试；对于 Work1 模式，需将从 Flash 将程序、数据搬运到 PSRAM，Debug 时，对 main 函数入口设置成硬断点，如图 4-16 所示。

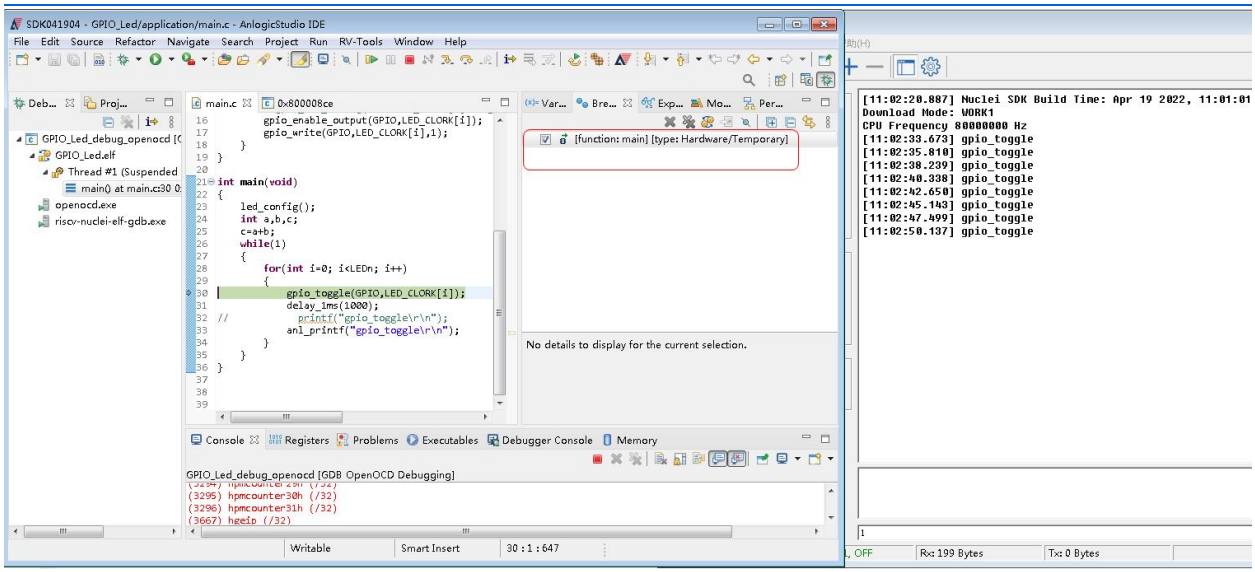

图 4-16 Debug GPIO_Led

Breakpoint 属性设置如下图 4-17、4-18 所示。

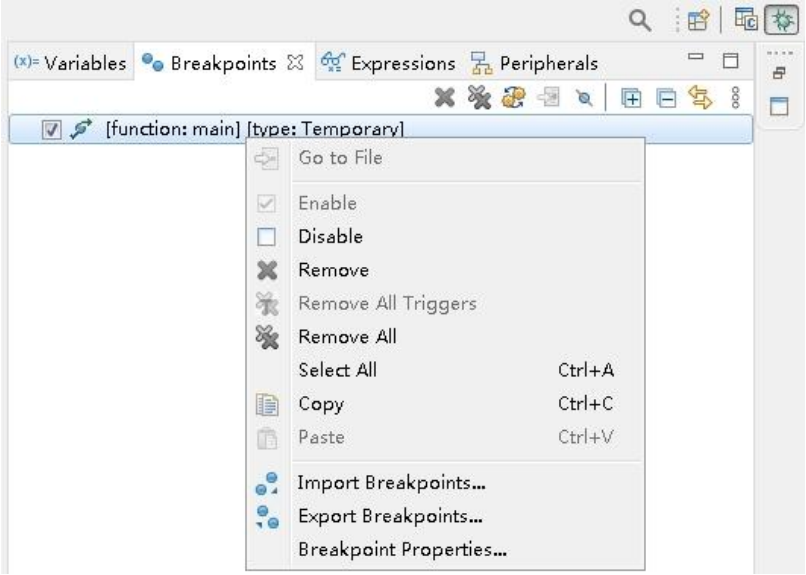

图 4-17 中断属性设置

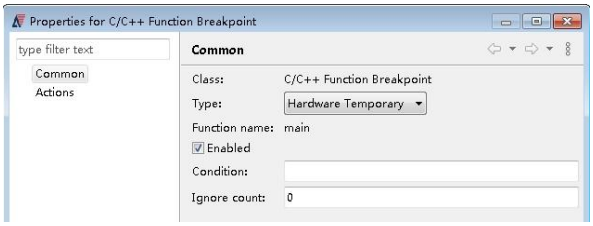

图 4-18 Type 选择

5 位流与固件更新

通过 TD Download 界面，下载 Hex 文件，如图 5-1 所示。

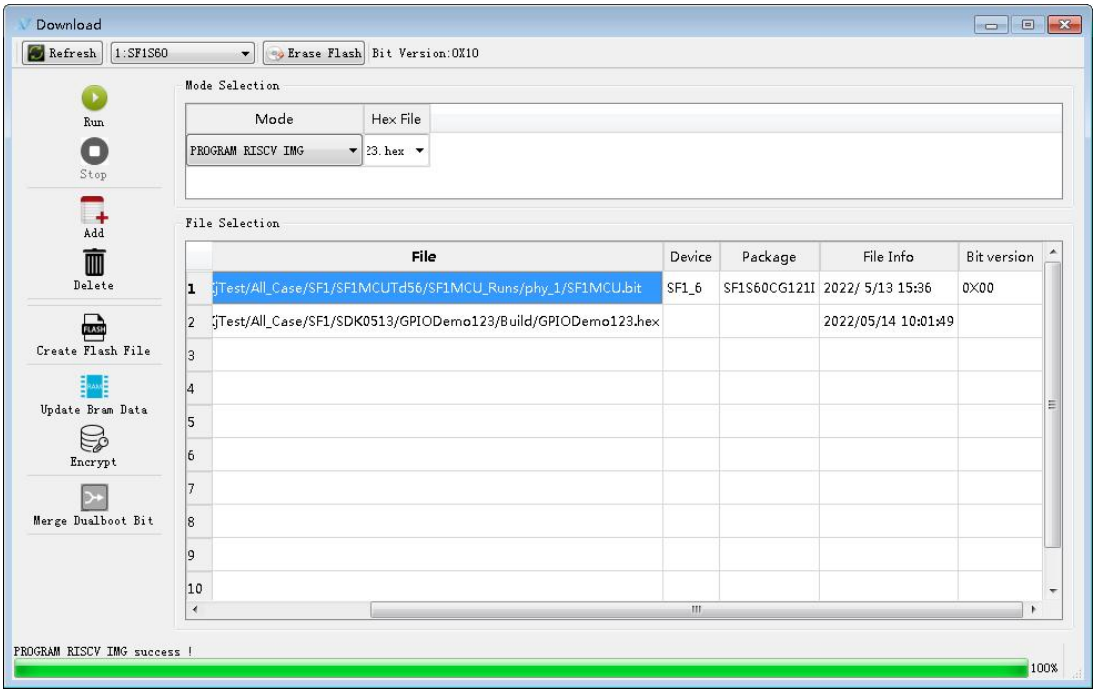

图 5-1 Hex 文件下载

点击 Add，分别添加 bit 文件、hex 文件。然后选择 PROGRAM RISC IMG,选择对应的 hex 文件，选中对应的 bit 文件，点击 Run 按钮，可将 bit 和 hex 文件烧写到 Flash 上。

版本信息

日期	版本	修订记录
2022/4/20	1.0	版本初始化
2022/5/24	1.1	增加 SF1 Demo 板开发环境描述 优化 4.3 AnlogicStudio 工程创建与调试 部分描述 增加 5 位流与固件更新 部分描述

版权所有©2022 上海安路信息科技股份有限公司

未经本公司书面许可，任何单位和个人都不得擅自摘抄、复制、翻译本文档内容的部分或全部，并不得以任何形式传播。

免责声明

本文档并未授予任何知识产权的许可，并未以明示或暗示，或以禁止发言或其它方式授予任何知识产权许可。除安路科技在其产品的销售条款和条件中声明的责任之外，安路科技概不承担任何法律或非法律责任。安路科技对安路科技产品的销售和/或使用不作任何明示或暗示的担保，包括对产品的特定用途适用性、适销性或对任何专利权、版权或其它知识产权的侵权责任等，均不作担保。安路科技对文档中包含的文字、图片及其它内容的准确性和完整性不承担任何法律或非法律责任，安路科技保留修改文档中任何内容的权利，恕不另行通知。安路科技不承诺对这些文档进行适时的更新。